```
info2.rank +=
info1.rank;                                                    #10
 return true
```

#1 Takes two elements, merges their partitions and returns `true` iff the two elements were in two different partitions that now are merged, `false` if they were already in the same partition;
#2-#3 Retrieve the roots of the trees to which `elem1` and `elem2` belong to. If its argument is invalid or not found, this call will throw.
#4 Compare `r1` and `r2`, and if they are the same there is nothing left to do, the elements are already in the same partition, and so no merge happens: `false` is returned.
#5 At this point, we know we need to merge the two partitions, so it looks for the info nodes for both roots.
#6 Check if the first tree has a larger rank (more elements) or vice versa: the smallest tree will become a subtree of the root of the largest tree.
#7 | #9 Set the root of the smallest tree to the other root.
#8 | #10 Updates the rank of the root (of both trees, now). It’s not worth updating the rank of the other (former) root, because it will never be checked again.

We still retrieve the elements at the roots of the trees as before, and still check they are not the same.

But after that, we actually have to retrieve the info for both roots, and we check which tree is larger: the smaller one will end up as the child, and we will reassign its root. Moreover, we need to update the rank for the larger tree's root: its